

Certified program synthesis with a multi-modal verifier



Yueyang
Feng^{1,*}



Dipesh
Kafle^{1,*}



Vladimir
Gladshstein¹



Vitaly
Kurin²



George
Pîrlea¹



Qiyuan
Zhao¹



Peter
Müller³



Ilya
Sergey¹

¹National University of Singapore ²Neapolis University Pafos ³ETH Zurich

*Joint first authors

ASE '26 · October 15, 2026 · Munich, Germany

AI writes code for us. An idea becomes an implementation in seconds.

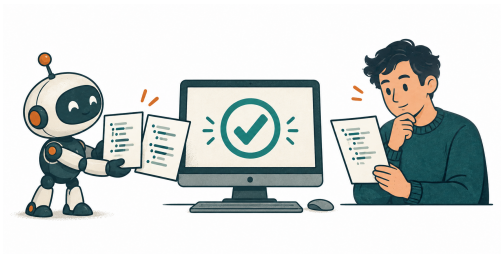


AI writes code for us. An idea becomes an implementation in seconds.



Who reviews all this code?

Generating code is fast. Establishing confidence still takes tremendous amount of work.



Ask the AI for a proof, too.

Humans only inspect the claim and assumptions;
proof checking is left to the verifier.

Fermat's Last Theorem

13 million lines of Lean;
machine-checked proof.

[Anthropic, September 2026](#)

Navier-Stokes

Proof released with a
Lean formalization.

[OpenAI, September 2026](#)

Can we automate the whole process?

English task

Describe what the program should do.



Runnable program

An implementation we can execute.

Formal specification

A precise statement of intended behavior.

Checked proof

The program satisfies the specification.

Why is this difficult?

Specifications are hard to get right

Capturing intent precisely is difficult, even for humans.

We found issues in existing verification benchmarks:

16 issues in VERINA

18 issues in CLEVER

Why is this difficult?

Specifications are hard to get right

Capturing intent precisely is difficult, even for humans.

We found issues in existing verification benchmarks:

16 issues in VERINA

18 issues in CLEVER

Proofs are hard to complete

Even for a correct program, proof search can consume many tokens and still leave goals unproved.

Easier, more efficient certified synthesis

Test

Reject incorrect candidates cheaply.

Automate

Discharge goals with SMT and Lean automation like **grind**.

Prove interactively

Handle goals automation cannot solve.

Easier, more efficient certified synthesis

Test

Reject incorrect candidates cheaply.

Automate

Discharge goals with SMT and Lean automation like **grind**.

Prove interactively

Handle goals automation cannot solve.

Combine *different modes* to reject incorrect candidates and prove correct ones.

Easier, more efficient certified synthesis

Test

Reject incorrect candidates cheaply.

Automate

Discharge goals with SMT and Lean automation like **grind**.

Prove interactively

Handle goals automation cannot solve.

Combine *different modes* to reject incorrect candidates and prove correct ones.

Our hypothesis: Using **Velvet**, a multi-modal verifier in Lean, we can certify more programs within the same budget.

```

method IsNonPrime (n : Nat) return (result : Bool)
  ensures result = true  $\leftrightarrow$   $\neg$ isPrime n
  do
    if n  $\leq$  1 then return true
    let mut i : Nat := 2
    let mut ret : Bool := false
    while i * i  $\leq$  n
      invariant  $\neg$ ret  $\leftrightarrow$   $\forall d, 2 \leq d \wedge d < i \rightarrow n \% d \neq 0$ 
      invariant i  $\geq$  2  $\wedge$  (i - 1) * (i - 1)  $\leq$  n
    do
      if n % i = 0 then ret := true
      i := i + 1
    return ret

prove_correct IsNonPrime by
  loom_solve -- automated VC discharge
  ...      -- remaining interactive proof

```

```
method IsNonPrime (n : Nat) return (result : Bool)
```

```
  ensures result = true  $\leftrightarrow$   $\neg$ isPrime n
```

```
  do
```

```
    if n  $\leq$  1 then return true
```

```
    let mut i : Nat := 2
```

```
    let mut ret : Bool := false
```

```
    while i * i  $\leq$  n
```

```
      invariant  $\neg$ ret  $\leftrightarrow$   $\forall d, 2 \leq d \wedge d < i \rightarrow n \% d \neq 0$ 
```

```
      invariant i  $\geq$  2  $\wedge$  (i - 1) * (i - 1)  $\leq$  n
```

```
    do
```

```
      if n % i = 0 then ret := true
```

```
      i := i + 1
```

```
    return ret
```

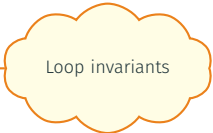
```
prove_correct IsNonPrime by
```

```
  loom_solve -- automated VC discharge
```

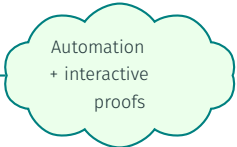
```
  ...      -- remaining interactive proof
```



Postcondition



Loop invariants



Automation
+ interactive
proofs

Running example

Determine whether an array is a rotation of a non-decreasing array.

Duplicates and zero rotation are allowed.



Not a rotation of a sorted array:



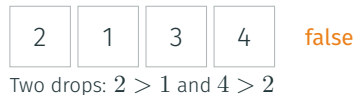
Running example

Determine whether an array is a rotation of a non-decreasing array.

Duplicates and zero rotation are allowed.



Not a rotation of a sorted array:



Accept if there is at most one cyclic drop.

Compare adjacent elements, including the last-to-first pair.

LeetProof orchestrates the entire pipeline from natural language to a verified implementation in the following phases:

1. Specification

Generate a specification and concrete test cases.

Type-check, review, and test the specification.

2. Program

Generate executable code and loop invariants.

Run tests and check verification conditions.

3. Proof

Prove the verification conditions.

Apply automation, then AI-assisted Lean proofs.

Feedback guides revisions within each stage.

Capturing intent is difficult

Capturing intent is difficult

- Writing precise specifications is difficult for humans and LLMs.

Capturing intent is difficult

- Writing precise specifications is difficult for humans and LLMs.
- A proof is only as good as its specification. So, it is very important that specification is validated against the intent somehow.

Capturing intent is difficult

- Writing precise specifications is difficult for humans and LLMs.
- A proof is only as good as its specification. So, it is very important that specification is validated against the intent somehow.

Key insight: We cannot prove agreement with informal intent, but we can effectively search for mismatches.

Checking the LLM's understanding

Task description  Formal specification + example inputs and outputs

Reuse examples from the problem statement and ask the LLM for more based on its interpretation (about ten in total).

Input	Expected output
[4, 1, 2, 3]	true
[2, 1, 3, 4]	false
[1, 1, 1]	true

Gaining confidence in the generated specification

Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Key insight: Search for counterexamples with property-based testing (PBT).

Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Key insight: Search for counterexamples with property-based testing (PBT).

Input validity

Does the input satisfy the precondition?

Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Key insight: Search for counterexamples with property-based testing (PBT).

Input validity

Does the input satisfy the precondition?

Expected output

Does the intended answer satisfy the postcondition?

Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Key insight: Search for counterexamples with property-based testing (PBT).

Input validity

Does the input satisfy the precondition?

Expected output

Does the intended answer satisfy the postcondition?

Output uniqueness

Does the specification also admit a different answer?

Keep the input fixed; sample alternative outputs by mutating the expected output.

Result

-- A drop is a strict decrease to the cyclic successor.

```
def isDrop (nums : Array Int) (i : Nat) : Prop := ...
```

-- A sorted-and-rotated array has at most one cyclic drop.

```
def rotSortedProp (nums : Array Int) : Prop := ...
```

-- No extra assumptions on the input.

```
def precondition (nums : Array Int) : Prop := True
```

-- The result exactly decides the sorted-and-rotated property.

```
def postcondition (nums : Array Int) (result : Bool) :=  
  result = true ↔ rotSortedProp nums
```

```
def test1_nums : Array Int := #[1, 2, 3]
```

```
def test1_Expected : Bool := true
```

-- Additional test cases omitted.

Result

-- A drop is a strict decrease to the cyclic successor.

```
def isDrop (nums : Array Int) (i : Nat) : Prop := ...
```

-- A sorted-and-rotated array has at most one cyclic drop.

```
def rotSortedProp (nums : Array Int) : Prop := ...
```

-- No extra assumptions on the input.

```
def precondition (nums : Array Int) : Prop := True
```

-- The result exactly decides the sorted-and-rotated property.

```
def postcondition (nums : Array Int) (result : Bool) :=  
  result = true ↔ rotSortedProp nums
```

```
def test1_nums : Array Int := #[1, 2, 3]
```

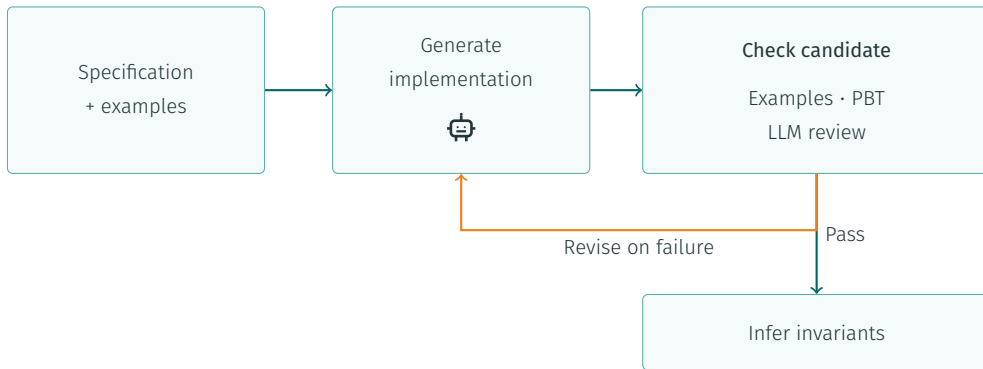
```
def test1_Expected : Bool := true
```

-- Additional test cases omitted.

If we had $\rightarrow$ instead of $\leftrightarrow$, an implementation returning **false** always would satisfy the spec.

PBT catches this with output uniqueness.

Program generation



Program

```
while i < n  
do  
  ...
```

We ask the LLM to propose invariants, then focus our effort on validating them.

Program

```
while i < n  
do  
  ...
```

LLM proposes
annotations



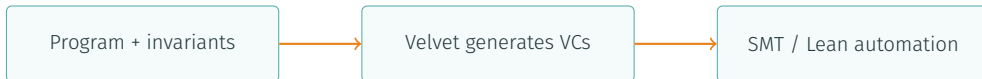
Annotated program

```
while i < n  
  invariant "inv_bounds" ( $i \leq n$ )  
  invariant "inv_drops_count" (...)  
  decreasing n - i  
do  
  ...
```

We ask the LLM to propose invariants, then focus our effort on validating them.

Gaining confidence in invariants: Prove with automation

Discharge obligations automatically using SMT / Lean's automation.



A simplified obligation for illustrative purpose

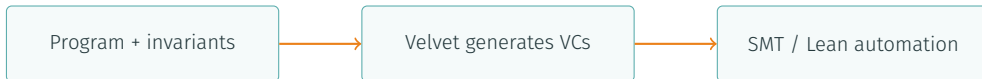
$i \ n : \text{Nat}$

$h : i < n$

$\vdash i + 1 \leq n \wedge n - (i + 1) < n - i$

Gaining confidence in invariants: Prove with automation

Discharge obligations automatically using SMT / Lean's automation.

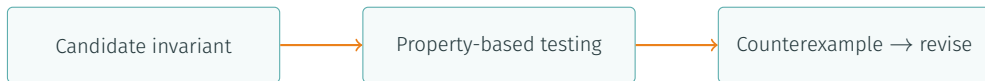


A simplified obligation for illustrative purpose

```
i n : Nat  
h : i < n  
⊢ i + 1 ≤ n ∧ n - (i + 1) < n - i
```

For the final program of our running example, 14/18 VCs were discharged using automation.

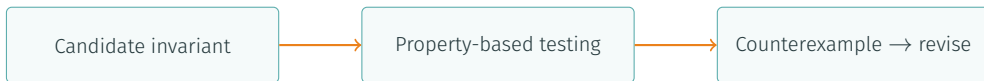
Gaining confidence in invariants: Counterexample search



Invariant from the running example, with `=` changed to `<`.

```
invariant "inv_drops_count" (drops = < (Finset.filter  
(fun k : Nat => nums[(k + 1) % n]! < nums[k]!)  
(Finset.range i)).card)
```

Gaining confidence in invariants: Counterexample search



Invariant from the running example, with `=` changed to `<`.

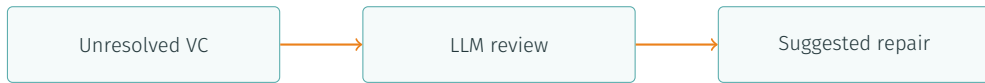
```
invariant "inv_drops_count" (drops = < (Finset.filter  
  (fun k : Nat => nums[(k + 1) % n]! < nums[k]!)  
  (Finset.range i)).card)
```

[velvet_plausible_test] FAIL:

invariant "inv_drops_count" doesn't hold

nums = #[-18, 10, -13, 11, 8, 17, 13, -19, 15, -1, -27, -25]

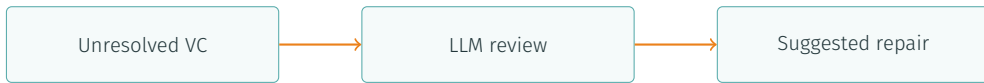
Gaining confidence in invariants: LLM judge



Simplified review request

```
{
  "goal": "theorem post ... : drops ≤ 1 ↔ rotSortedProp nums",
  "context": "Specification, implementation, current invariants",
  "prompt": [
    "Find counterexamples; check empty collections, zero, singletons.",
    "Attempt a proof sketch; name lemmas and identify blockers.",
    "Decide: provable, or hypotheses insufficient?"
  ]
}
```

Gaining confidence in invariants: LLM judge



Simplified review request

```
{
  "goal": "theorem post ... : drops ≤ 1 ↔ rotSortedProp nums",
  "context": "Specification, implementation, current invariants",
  "prompt": [
    "Find counterexamples; check empty collections, zero, singletons.",
    "Attempt a proof sketch; name lemmas and identify blockers.",
    "Decide: provable, or hypotheses insufficient?"
  ]
}
```

Illustrative feedback from the judge

```
{
  "is_provable": false,
  "justification": "Index bounds do not constrain drops.",
  "correction_hint": "Track cyclic decreases in the visited prefix."
}
```

```
1 method CheckSortedAndRotated (nums : Array Int)
2 return (result : Bool)
3 require precondition nums
4 ensures postcondition nums result
5 do
6   let n := nums.size
7   if n ≤ 1 then return true
8   else
9     let mut drops : Nat := 0
10    let mut i : Nat := 0
11    while i < n
12      invariant "inv_bounds" (i ≤ n)
13      invariant "inv_n_def" (n = nums.size)
14      invariant "inv_n_pos" (n > 0)
15      invariant "inv_drops_count" (drops = (Finset.filter
16        (fun k : Nat => nums[(k + 1) % n]! < nums[k]!)
17        (Finset.range i)).card)
18      decreasing n - i
19    do
20      let a := nums[i]!
21      let b := nums[(i + 1) % n]!
22      if b < a then drops := drops + 1
23      i := i + 1
24    return (drops ≤ 1)
25
26 #guard (CheckSortedAndRotated #[4, 1, 2, 3]).extract
27 velvet_plausible_test CheckSortedAndRotated
```

Program + invariant

```
1 method CheckSortedAndRotated (nums : Array Int)
2 return (result : Bool)
3 require precondition nums
4 ensures postcondition nums result
5 do
6   let n := nums.size
7   if n ≤ 1 then return true
8   else
9     let mut drops : Nat := 0
10    let mut i : Nat := 0
11    while i < n
12      invariant "inv_bounds" (i ≤ n)
13      invariant "inv_n_def" (n = nums.size)
14      invariant "inv_n_pos" (n > 0)
15      invariant "inv_drops_count" (drops = (Finset.filter
16        (fun k : Nat => nums[(k + 1) % n]! < nums[k]!))
17        (Finset.range i)).card)
18      decreasing n - i
19    do
20      let a := nums[i]!
21      let b := nums[(i + 1) % n]!
22      if b < a then drops := drops + 1
23      i := i + 1
24    return (drops ≤ 1)
25
26 #guard (CheckSortedAndRotated #[4, 1, 2, 3]).extract
27 velvet_plausible_test CheckSortedAndRotated
```

Loop invariant
annotations

Unit test using
a concrete
test case

Property-based
testing (PBT)

How accurate are LeetProof's specifications compared with human-written references?

188 VERINA problems; equivalence checking and manual review.

How accurate are LeetProof's specifications compared with human-written references?

188 VERINA problems; equivalence checking and manual review.

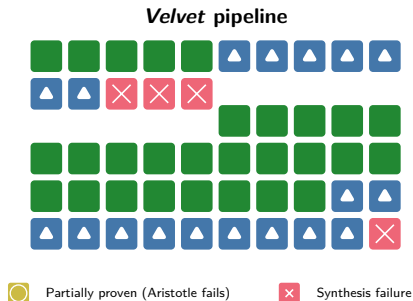
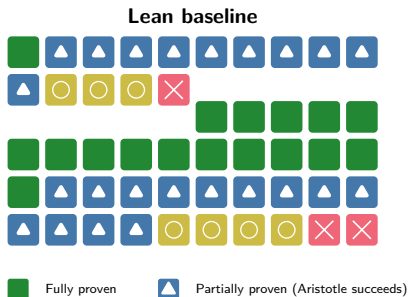
97.4%

150 equivalent to reference + 33 defensible*

* Defensible: justified differences due to reference errors, valid precondition variations, or ambiguous descriptions.

More certified solutions than single-mode Lean, at the same cost

Fully proven: Lean 17/50; Velvet 28/50.



RQ3: Closing the remaining proofs

Can additional proof effort close the remaining goals?

Partially proven programs completed by Aristotle:

Lean 23 / 30

Velvet 18 / 18

All partially proven Velvet programs were certified.

RQ4: Across different LLMs

Does the advantage hold with another model?

Fully proven programs at \$5 per problem:

	Lean	Velvet
GPT-5.2	17/50	28/50
Opus 4.6	7/25	10/25

Opus: a random subset of 25 problems.

Velvet leads under both models.

What have we seen

What have we seen

- PBT is highly effective.

What have we seen

- PBT is highly effective.
- Decomposition simplifies synthesis.

What have we seen

- PBT is highly effective.
- Decomposition simplifies synthesis.
- Multi-modality pays off.

What have we seen

- PBT is highly effective.
- Decomposition simplifies synthesis.
- Multi-modality pays off.

Interesting future directions

- Certified synthesis of protocols and distributed systems.

What have we seen

- PBT is highly effective.
- Decomposition simplifies synthesis.
- Multi-modality pays off.

Interesting future directions

- Certified synthesis of protocols and distributed systems.
- Going Beyond functional correctness.

What have we seen

- PBT is highly effective.
- Decomposition simplifies synthesis.
- Multi-modality pays off.

Interesting future directions

- Certified synthesis of protocols and distributed systems.
- Going Beyond functional correctness.

Follow our work: verse-lab.org

[Velvet](#) · [Artifact](#) · [Paper](#)

Thank you!